

LADO B

Portal de noticias con IA

Docker Compose · NGINX · Round Robin · Node.js 22

ESTUDIANTE

Enner Esaí Mendizabal Castro

CARNÉ

202302220

CURSO

Software Avanzado

CATEDRÁTICO

ING. MARCO TULIO ALDANA PRILLWITZ

AUXILIAR

KEVIN RAÚL POZUELOS ESTRADA

ACCESO

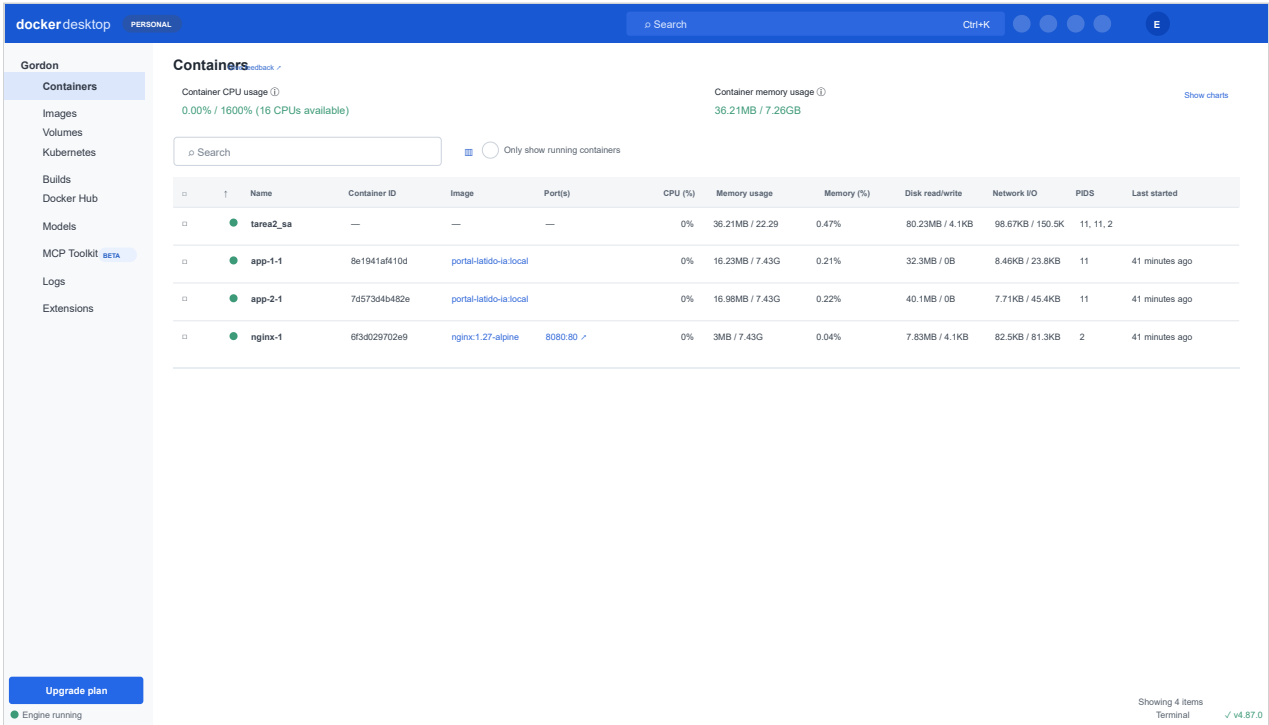
<http://localhost:8080>

Nota académica.

Las noticias fueron generadas o asistidas por ChatGPT y revisadas para este ejercicio. No son información periodística verificada.

Los servicios están funcionando

La evidencia visual muestra las dos instancias de la aplicación y NGINX. El puerto público pertenece únicamente al proxy.



Estado visual basado en la captura de Docker Desktop compartida y contrastado con Docker Compose.

```
docker compose ps
```

NAME	SERVICE	STATUS	PORTS
app-1-1	app-1	Up (healthy)	3000/tcp
app-2-1	app-2	Up (healthy)	3000/tcp
nginx-1	nginx	Up	0.0.0.0:8080->80/tcp

PRIVACIDAD

Las apps no publican el puerto 3000 en el host.

ENTRADA ÚNICA

El navegador entra por NGINX mediante el puerto 8080.

El portal entra por NGINX

Dirección de acceso: <http://localhost:8080>. La interfaz muestra seis noticias y la instancia que respondió.



Portada del portal en escritorio.

Encabezados

```
HTTP/1.1 200 OK
X-App-Instance: app-1
X-Load-Balancer: nginx
Content-Type: text/html; charset=utf-8
```

Rutas verificadas

RUTA	RESULTADO
/	Portada · 200
/noticias	Catálogo · 200
/noticias/:slug	Detalle · 200
/api/noticias	JSON · 200
/nginx-status	Estado · 200

Contenido completo y atribuido



Noticia individual con título, fecha, categoría, imagen, resumen, contenido y ficha editorial.

SEIS NOTICIAS

La API devuelve seis elementos con título, fecha, categoría, imagen y contenido.

HERRAMIENTA IDENTIFICADA

Cada noticia muestra que fue generada con ChatGPT y revisada por el equipo.

Responsabilidad.

El portal advierte que el contenido es académico y no información verificada. Los prompts están en `prompts/noticias.md`.

Responsive



Prueba Playwright a 390 × 844, sin overflow horizontal.

Round Robin entre dos instancias

NGINX distribuye solicitudes mediante el upstream `newsroom_app` . El encabezado `X-App-Instance` demuestra qué backend respondió.

```
sh scripts/verify.sh • solicitudes consecutivas
```

```
Solicitud 01 -> app-1
Solicitud 02 -> app-2
Solicitud 03 -> app-1
Solicitud 04 -> app-2
Solicitud 05 -> app-1
Solicitud 06 -> app-2
Solicitud 07 -> app-1
Solicitud 08 -> app-2
Solicitud 09 -> app-1
Solicitud 10 -> app-2
Solicitud 11 -> app-1
Solicitud 12 -> app-2
```

BACKEND	MARCA VISUAL	ENCABEZADO
app-1:3000	LADO A • LA	X-App-Instance: app-1
app-2:3000	LADO B • LB	X-App-Instance: app-2

Algoritmo: Round Robin.

NGINX reparte solicitudes secuencialmente entre los backends disponibles.

El portal continúa si una instancia se detiene

Se detuvo `app-1` sin detener NGINX. El tráfico continuó siendo atendido por `app-2`.

`docker compose stop app-1` • resultado

```
app-1-1          Stopped
app-2-1          Up (healthy)
nginx-1          Up • 0.0.0.0:8080->80/tcp
```

```
Solicitud 01 -> app-2
Solicitud 02 -> app-2
Solicitud 03 -> app-2
Solicitud 04 -> app-2
Solicitud 05 -> app-2
Solicitud 06 -> app-2
```

Resultado: el portal, las rutas y la API continuaron respondiendo.

Configuración

```
max_fails=2
fail_timeout=5s
proxy_next_upstream error timeout
http_502 http_503 http_504
```

Recuperación

```
docker compose start app-1
docker compose ps

app-1 volvió a healthy.
```

Configuraciones entregadas

Los archivos completos están en el repositorio. Estos fragmentos muestran las partes evaluables.

DOCKER-COMPOSE.YML

```
services:
  app-1:
    build: .
    environment:
      INSTANCE_ID: app-1
    expose: ["3000"]
    networks: [newsroom]

  app-2:
    environment:
      INSTANCE_ID: app-2
    expose: ["3000"]
    networks: [newsroom]

  nginx:
    image: nginx:1.27-alpine
    ports: ["8080:80"]
    networks: [newsroom]
```

NGINX/NGINX.CONF

```
upstream newsroom_app {
    server app-1:3000 max_fails=2 fail_timeout=5s;
    server app-2:3000 max_fails=2 fail_timeout=5s;
}

location /noticias {
    proxy_pass http://newsroom_app;
    proxy_set_header X-Load-Balancer "nginx";
    proxy_next_upstream error timeout http_502 http_503 http_504;
}
```

DOCKERFILE

```
FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production PORT=3000
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY package.json ./
COPY server.js ./
COPY public ./public
```



```
USER appuser
EXPOSE 3000
HEALTHCHECK ...
CMD ["node", "server.js"]
```

Material preparado

- ✓ Código fuente completo
- ✓ docker-compose.yml
- ✓ Recursos SVG locales
- ✓ Prompts conservados
- ✓ Arquitectura documentada
- ✓ Guion del video
- ✓ Dockerfile
- ✓ Configuración de NGINX
- ✓ README.md
- ✓ Scripts de verificación
- ✓ PDF de evidencias
- ✓ Enlace de video pendiente

Demostración final.

Ejecutar `docker compose up --build -d`, abrir `http://localhost:8080`, mostrar `docker compose ps`, ejecutar el script, detener `app-1`, comprobar el failover, restaurarlo y mostrar `prompts/noticias.md`.

ARCHIVO	QUÉ CONTIENE
server.js	Servidor HTML/JSON, seis noticias y marca por instancia.
docker-compose.yml	Dos apps, NGINX, red interna y puerto público.
nginx/nginx.conf	Upstream, proxy, headers, rutas y failover.
prompts/noticias.md	Prompts editoriales y bitácora del desarrollo.
docs/evidencias/evidencias.pdf	Este informe formal.

Pendiente externo: grabar y subir el video demostrativo de 3–5 minutos.